JET - Take Home Assignment

Ani Dharmarajan
Jan 4, 2025

Notes/Assumptions

- Model objective
  - Predict number of orders (order busyness) per h3-day-hour. Assuming that this model will be used to make real time predictions of order busyness at any given time, based on courier location and restaurant location.
- Encoding
  - Used ordinal encoding with explicitly specified columns (rather than encoding all object/category columns)
  - Decision Tree model should be configured to treat h3_index as a categorical column, since the encoded labels represent discrete categories rather than ordinal values?
  - In notebook, should encoding happen after train-test split?
- Model Packaging
  - In production in the future, use sklearn pipeline object to bundle encoder and model together for unified versioning
- Centroids
  - Saved as DataFrame during training, loaded during inference
- Training
  - Refits on full dataset after hyperparameter tuning
- Testing
  - Implemented 2 unit tests and 1 integration test (limited scope for assignment)
- Configuration
  - Used single .py config file for simplicity. In production, use separate YAML configs per environment?
- Additional notes as TODOs left in code

System Design Assumptions

- Batch Training
  - In production, model training can be based on a schedule or triggered based on drift detection
- Real-Time Inference
  - Assuming real-time inference, because features used in model can only be obtained in real time (i.e. courier location)
  - Predictions served via always-on endpoint (not batch predictions)

Tools & services used for productionization

- Google Cloud Composer
    - Training pipeline orchestration
    - Airflow & Kubernetes
- Google Artifact Registry
    - Stores Docker images for training and inference containers
- Google Cloud Storage
    - Stores raw data, processed data, and model artifacts
- Vertex AI Model Registry
    - Tracks model versions and metadata, references artifacts in GCS
- Vertex AI Endpoint
    - Hosts real-time inference with custom Docker container
- GitHub Actions
    - CI/CD for building and pushing Docker images, configuring composer env, deploying to endpoint

Full productionization next steps

- Data Processing
    - Add missing value checks and imputation
    - Maybe add feature scaling for future proofing (not necessary here because it is a decision tree model)
    - Implement data drift monitoring (scheduled runs)
- Model Training
    - Read training data from BigQuery
    - Log training metrics with artifact (Vertex AI)
    - Use sklearn Pipeline class objects instead of functions
    - Trigger retraining based on concept drift/data drift
    - Maintain golden dataset to compare challenger vs champion model performance, and promote models accordingly
- Inference
    - Integrate real-time data ingestion (e.g., Kafka?) or accept features directly from client
    - Implement fallback model/predictions if inference fails

Code quality notes

- Pytest
  - uUnit and integration testing
- Logging
  - Logging for debugging and monitoring
- Memory profiling & run time
  - Log memory and run time
- Other (partially implemented)
  - Schema validation: Input/output validation for data integrity
  - Try Except blocks
  - Type hinting
  - Docstrings

SYSTEM ARCHITECTURE